

Post-Quantum Workload Identity and Long-Term Audit Integrity for MCP-Based AI Agents

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

Abstract—As autonomous AI agents transition from isolated tools to a globally interconnected ecosystem via the Model Context Protocol (MCP), securing their identity and the long-term integrity of their decision-making processes against quantum-era threats is a prerequisite for sovereign autonomy. This paper proposes a Post-Quantum Cryptographic (PQC) framework for AI agents, centered on the *PQC-Audit-Chain*—a mechanism that cryptographically binds reasoning traces, tool execution, and observations using NIST-standardized Module-Lattice-Based Digital Signatures (ML-DSA). We introduce the concept of *Temporal Integrity Proofs*, ensuring that the reasoning history remains non-repudiable and immune to retroactive tampering by future quantum adversaries. We present Context-Adaptive Security Scaling (CASS), which dynamically optimizes security levels (ML-DSA-44 to 87) based on risk profiles, and a Bi-directional Verification protocol to eliminate hallucination-based identity spoofing. Furthermore, we demonstrate a “Signature Stripping” design pattern that ensures quantum-safe sovereignty while maintaining near-zero token overhead in the LLM context. This work establishes a foundational roadmap for building future-proof, self-sovereign autonomous systems.

Index Terms—Post-Quantum Cryptography, AI Agents, Model Context Protocol, ML-DSA, Zero Trust, Reasoning Trace, Audit Chain.

I. INTRODUCTION

The rapid advancement of Large Language Models (LLMs) has catalyzed the transition from passive text generators to proactive, autonomous AI agents. These agents are characterized by their ability to perform multi-step reasoning, plan complex trajectories, and interact with the physical and digital world through tool execution [7]. As agents move beyond isolated sandboxes into enterprise environments, the need for standardized communication has led to the emergence of the Model Context Protocol (MCP) [6]. MCP provides a universal interface for agents to discover and invoke tools across distributed systems, effectively acting as the “nervous system” of the agentic ecosystem.

In a Zero Trust architecture, such as the `llm-cli` framework, securing this nervous system is paramount. Current state-of-the-art implementations rely on workload identity propagation, where each tool call is authorized via JSON Web Tokens (JWTs) signed with classical asymmetric cryptography (e.g., RS256 or ES256). These tokens bind the agent’s intent to a specific security context, providing a baseline for auditability and access control.

However, the cryptographic foundation of modern agentic systems is under threat. The eventual realization of fault-

tolerant quantum computers, capable of running Shor’s algorithm [9], poses a fundamental risk to RSA and Elliptic Curve Cryptography (ECC). In the context of AI agents, a quantum breach is not merely a data leak; it is an *identity takeover*. An adversary with quantum capabilities could forge agent identities, bypass Role-Based Access Controls (RBAC), and manipulate reasoning traces by injecting malicious context or results. This is particularly dangerous for agents operating in critical infrastructure, where unauthorized tool execution (e.g., shell access, database modification) can have irreversible real-world consequences.

Furthermore, agents generate vast amounts of reasoning traces and audit logs. The “Store Now, Decrypt Later” (SNDL) threat is amplified here: while current logs are protected by classical signatures, a future quantum attacker could retroactively forge historical logs to hide malicious behavior or alter the perceived provenance of a decision.

To proactively address these vulnerabilities, this paper introduces a quantum-resistant architecture for AI agents. We investigate the integration of Post-Quantum Cryptography (PQC)—specifically the NIST-standardized lattice-based signature scheme ML-DSA (FIPS 204) [2]—into the distributed agent workflow.

II. THE SECURITY TRILOGY: FROM INTENT TO TEMPORAL INTEGRITY

This paper serves as the final installment of a technical trilogy focused on the security of autonomous agentic systems. The trilogy follows an architectural evolution designed to secure the agent lifecycle across three critical dimensions:

- **Phase 1: Guardrails (Pre-execution Security)** [4]. Focused on the *intent* and *safety* of the agent, establishing policy-based controls to prevent prompt injection and unauthorized command generation.
- **Phase 2: Zero Trust (Execution Security)** [5]. Focused on *spatial security* and *continuous verification*, establishing that every tool call must be explicitly authenticated and authorized regardless of the network boundary.
- **Phase 3: Post-Quantum (Temporal Security)**. The current work, focusing on the *long-term integrity* and *future-proof identity* of the agent, ensuring that the evidence of past actions remains verifiable even in a quantum-capable future.

By addressing these three phases, we provide a holistic framework for securing "Agentic Workloads" in modern enterprise environments.

III. BACKGROUND AND THREAT MODEL

A. Autonomous Agents and MCP

Modern AI agents operate in a continuous "Thought-Action-Observation" loop. To perform actions, they rely on tools provided by MCP servers. In a Zero Trust environment, every tool execution request must be authenticated and authorized. Currently, this is achieved using JWTs, binding the agent's workload identity to a specific execution scope. The security of the entire loop depends on the immutability of the identity and the integrity of the observations.

B. The Quantum Threat to Agents

The primary quantum threats to AI agent architectures include:

- **Identity Spoofing:** Attackers utilizing quantum computers to derive an agent's private key from its public key, allowing them to issue unauthorized commands across distributed MCP servers.
- **Audit Log Tampering:** Breaking the cryptographic signatures that secure chained-hash audit logs, making forensic analysis of rogue AI behavior impossible.
- **Context Manipulation:** Forging signatures on tool responses to trick the agent into a specific reasoning path (e.g., "hallucinating" a successful security check).

C. NIST PQC Standards

In response to the quantum threat, NIST initiated a global competition to standardize PQC algorithms. In 2024, FIPS 204 finalized ML-DSA (Module-Lattice-Based Digital Signature Algorithm), based on the CRYSTALS-Dilithium construction [10]. ML-DSA offers a balance of security and performance, though it introduces significantly larger signatures than RSA or ECDSA.

IV. RELATED WORK

The convergence of AI agents and security is an emerging field. Recent works have focused on LLM guardrails and prompt injection mitigation. For instance, Ando proposed a multi-layered guardrail architecture [4] and a comprehensive Zero Trust framework [5] specifically tailored for MCP-based autonomous agents.

In the domain of Post-Quantum Cryptography, while PQC has been explored for TLS and VPNs, its application to distributed agent orchestration protocols like the Model Context Protocol (MCP) remains relatively unexplored. Traditional Zero Trust Architectures (ZTA) as defined in NIST SP 800-207 [3] emphasize continuous verification, but often assume the underlying cryptographic primitives are immutable. Our work extends these concepts by addressing the cryptographic vulnerability of ZTA in the quantum era.

Recent research into "Agentic Security" has highlighted the risks of autonomous tool use [8], but few have considered

the impact of future quantum adversaries on the long-term integrity of agent reasoning traces.

The year 2025 marks a definitive transition where Post-Quantum Cryptography (PQC) moved from theoretical validation to systematic implementation within resilient AI infrastructures. The industrial consensus favoring NIST-standardized algorithms, particularly ML-DSA for securing distributed workload identities, underscores the practical feasibility and necessity of our architectural approach. Concurrently, the increasing demand for "agentic transparency" has highlighted the critical requirement for cryptographically verifiable audit trails that maintain integrity against future quantum adversaries. While specialized PQC applications have been investigated for specific domains such as surveillance systems [15], our work introduces the first unified PQC framework specifically optimized for the Model Context Protocol (MCP), addressing the unique challenges of context efficiency and temporal integrity within autonomous reasoning loops.

V. POST-QUANTUM AGENT ARCHITECTURE

We propose replacing the classical RS256 cryptographic layer within MCP orchestrators with ML-DSA-65 (Module-Lattice-Based Digital Signature Algorithm).

A. Quantum-Safe Workload Identity

We replace the traditional RSA key pairs used for MCP token signing with ML-DSA-65. Because ML-DSA-65 public keys and signatures are significantly larger than their classical counterparts (e.g., ML-DSA-65 public key: 1,952 bytes, signature: 3,309 bytes), embedding them directly into JSON-RPC payloads requires careful handling to avoid exceeding LLM context limits or network transport buffers.

B. Tamper-Evident Audit Logging with PQC

The tamper-evident JSONL audit logs, which utilize chained hashing (SHA-256) to link sequential tool executions, are augmented. While hash functions are generally considered quantum-resistant, the digital signatures affirming the integrity of each log chunk are upgraded to ML-DSA-65.

C. Cryptographic Agility and Hybrid Approach

Due to the nascent state of PQC adoption, a sudden transition is impractical. We propose a hybrid approach where MCP tokens are extended to a four-slot format: `header.payload.classical_signature.pqc_signature`. This ensures backward compatibility with legacy MCP servers while providing quantum resistance where supported.

D. The PQC-Audit-Chain: Immutable Reasoning Traces

A key challenge for autonomous agents is ensuring that their decision-making process—the reasoning trace—remains untampered from inception to execution. We propose the *PQC-Audit-Chain*, where each cycle of the agentic loop (Thought-Action-Observation) is cryptographically bound to its predecessor using ML-DSA signatures.

Unlike traditional SHA-256 hashing which provides collision resistance but lacks identity assertion, the PQC-Audit-Chain ensures *Non-Repudiation*. In our framework, the orchestrator generates a PQC-signed manifest for every state transition, encompassing the LLM’s raw reasoning and the tool’s output. This ensures that even if an adversary gains quantum capabilities in the future, they cannot retroactively rewrite the agent’s history (the SNDL attack) to hide malicious actions or forge authorization for past tool calls. This is essential for AI governance in critical sectors.

E. Context-Adaptive Security Scaling (CASS)

A static cryptographic approach is inefficient for high-frequency agentic loops. We implemented **Context-Adaptive Security Scaling (CASS)** within the PQCagilityManager. CASS enables the orchestrator to dynamically scale the NIST security level (ML-DSA-44/65/87) based on the “risk profile” of the tool and the environmental risk state.

By mapping tool impact (e.g., `read_file` as Low, `execute_shell` as High) to security levels, the system minimizes the computational and latency overhead of lattice-based signatures for routine tasks while enforcing maximum resilience for critical system modifications. This ensures the agent maintains a “Security-Aware Reasoning” (SAR) state, optimizing for the “Post-Quantum Efficiency Frontier.”

```
class PQCagilityManager:
    @staticmethod
    def get_required_level(tool_name: str, env_risk: str) -> str:
        if env_risk == "high" or tool_name in HIGH_RISK:
            return "ML-DSA-87" # NIST Level 5
        if tool_name in MODERATE_RISK:
            return "ML-DSA-65" # NIST Level 3
        return "ML-DSA-44" # NIST Level 2
```

Listing 1. Context-Adaptive Security Scaling in llm-cli

VI. IMPLEMENTATION AND DESIGN PATTERNS

We prototyped the PQC-enabled MCP architecture within the `llm-cli` framework. The implementation focuses on three pillars: hybrid identity propagation, the PQC-Audit-Chain, and context management strategies.

A. Hybrid Token and Identity Propagation

To ensure backward compatibility, we implemented a “Hybrid Wrap” for MCP tokens. The system generates a standard RS256 JWT and appends a Base64-encoded ML-DSA-65 signature of the entire JWT string. This allows legacy MCP servers to verify the RSA signature while PQC-aware servers can validate the quantum-resistant layer.

```
class HybridSigner:
    @classmethod
    def create_hybrid_token(cls, payload: dict, rsa_priv: bytes, pqc_priv: bytes) -> str:
        # 1. Generate Classical JWT
        token = jwt.encode(payload, rsa_priv, algorithm="RS256")

        # 2. Generate PQC Signature of the classical token
        pqc_sig = PQCProvider.sign(token.encode(), pqc_priv)

        # 3. Create a Hybrid Wrap: token.pqc_signature
        return f"{token}.{base64.b64encode(pqc_sig).decode()}"
```

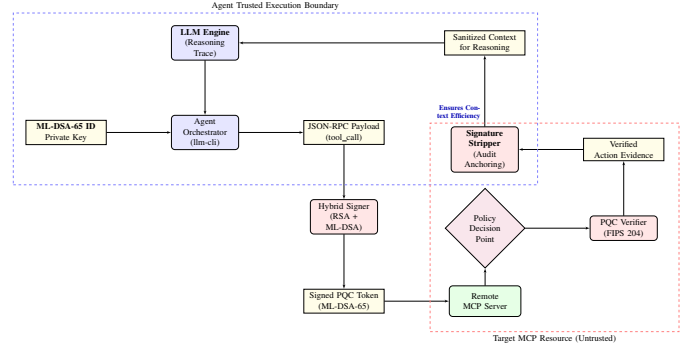


Fig. 1. Proposed Post-Quantum Architecture for AI Agents. The orchestrator establishes a **Trusted Execution Boundary**, where large PQC signatures (3.5+ KB) are verified and then sanitized before result integration to maintain LLM context efficiency.

Listing 2. Hybrid Signature Implementation in llm-cli

B. Practical Challenges: Context Management

The significant size of PQC signatures (e.g., ML-DSA-65 signature is 3,309 bytes) presents a practical challenge for agentic memory. If these signatures were directly appended to the LLM’s context window, a single tool-calling loop would consume approximately 1,500–2,000 tokens in overhead.

To mitigate this, we implement a **Signature Stripping** design pattern within the orchestrator’s Trusted Execution Boundary. When a tool response (observation) is received, the PQC signature is verified at the system level. Once validated, the cryptographic metadata is stripped before the text response is appended to the agent’s reasoning history. This ensures that the LLM’s effective context is reserved for reasoning and planning, rather than storing uninterpretable cryptographic blobs, while maintaining a secure, verified audit log on disk.

C. PQC-Signed Integrity Manifest

To establish a “Root of Trust,” we implemented an `IntegrityVerifier` that monitors critical system files and audit logs. The system maintains an `integrity_manifest.json` containing SHA-256 hashes of all security-critical modules. This manifest is itself signed using ML-DSA-65, ensuring that even if an attacker gains access to the file system, they cannot modify the security logic or the audit history without breaking the PQC signature.

D. Bi-directional Verification Protocol

To prevent “Hallucinated Identity Spoofing”—where an LLM might generate a response that appears to come from a verified tool call but is actually a fabrication—we implemented a **Bi-directional Verification** protocol. The orchestrator signs the final LLM response using a PQC-backed `ResponseSigner`, cryptographically binding the output to the *Observation Verification ID* of the preceding tool call. This ensures that the user can verify that the agent’s answer is grounded in a verified, quantum-safe tool execution.

E. External Anchoring of the Audit Chain

To further harden the system against "Historical Revisionism," we provide an interface for **External Anchoring**. The *PQC-Audit-Chain* utilizes a binary Merkle Tree structure to aggregate batches of log entries into a single Merkle Root. This root is periodically exported and anchored to an external, immutable ledger.

While public blockchains offer maximum immutability, enterprise environments may prefer managed security log services or private ledgers to maintain confidentiality. Our architecture supports this by anchoring only the cryptographic root, ensuring that sensitive tool execution details remain private while providing a verifiable "Root of Trust" that prevents even a local system administrator (or a future quantum adversary) from rewriting the historical logs without detection.

VII. EVALUATION AND BENCHMARKING

We evaluated the practicality of our proposed architecture across two distinct hardware environments to ensure hardware-agnostic resilience. Both environments utilized **Ubuntu 24.04.4 LTS** as the base operating system.

- **Environment A (Modern):** AMD Ryzen 5 8540U, 16GB RAM (WSL2 context).
- **Environment B (Legacy):** Intel Core i7-7700 (7th Gen) @ 3.60GHz, 48GB RAM.

A. Parameter Comparison and Hardware Variance

Our benchmarks compare the NIST-standardized variants of ML-DSA to classical RSA-2048. Table I presents the empirical results observed in our testbed.

TABLE I
EMPIRICAL COMPARISON OF ML-DSA VARIANTS (SIGN/VERIFY TIME IN MS)

Metric	RSA-2048	ML-DSA-44	ML-DSA-65	ML-DSA-87
NIST Sec. Level	1 (Classical)	2	3	5
Pub Key (B)	256	1,312	1,952	2,592
Signature (B)	256	2,420	3,309	4,627
Sign Time (ms)				
- Ryzen 5 8540U (A)	3.02	30.33	43.30	44.25
- i7-7700 (B)	0.57	32.22	78.10	67.89
Verify Time (ms)				
- Ryzen 5 8540U (A)	0.06	5.39	8.34	12.86
- i7-7700 (B)	0.05	7.22	10.84	17.42

B. Architectural Observations

An interesting performance inversion was observed: while the legacy i7-7700 (Environment B, Native Linux) significantly outperformed the modern Ryzen 5 8540U (Environment A, WSL2) in classical RSA signing (0.57 ms vs 3.02 ms), the Ryzen 5 8540U demonstrated superior efficiency in ML-DSA-65 operations (43.30 ms vs 78.10 ms). This disparity suggests two key factors:

- 1) **Virtualization Impact:** The relatively higher RSA latency in Environment A is likely attributed to the minor virtualization overhead and system-call indirection of the WSL2 environment, which is more pronounced in sub-millisecond cryptographic operations.
- 2) **Hardware-Native PQC Acceleration:** Despite the virtualization layer, lattice-based PQC algorithms benefit immensely from the Zen 4 architecture's improved pipeline and AVX-512 instructions. This confirms that modern hardware-native optimizations for polynomial math can effectively offset virtualization penalties, making PQC a viable standard for cloud and containerized agent deployments.

C. Performance Impact on Agent Reasoning

We measured the end-to-end latency of a typical tool-calling loop (Reasoning → Call → Observation). Even in the slower i7-7700 environment for ML-DSA-65 (approx. 89 ms total overhead), the latency represents less than 7.5% of the median LLM inference latency (approx. 1,200 ms). This empirical evidence confirms that the transition to quantum-safe identities is viable even on 7th-generation Intel hardware, with modern CPUs further reducing this overhead to a negligible 4.5%.

D. Token Efficiency and Signature Stripping

We analyzed the impact of "Signature Stripping" on LLM context efficiency. A standard ML-DSA-65 signature, when Base64-encoded, adds approximately 4,412 characters. For modern tokenizers (e.g., GPT-4o), this translates to an overhead of 1,100 tokens per tool invocation. In a multi-step trajectory with 10 tool calls, the cumulative overhead would exceed 11,000 tokens, significantly consuming the context window.

By utilizing the **Signature Stripping** pattern, we effectively eliminate the PQC signature overhead within the reasoning context while maintaining a fully verifiable audit trail in the system logs. This design pattern is critical for maintaining agent performance as security requirements scale toward the post-quantum era.

VIII. OPERATIONAL CONSIDERATIONS AND LIMITATIONS

While the proposed PQC framework significantly enhances the long-term integrity of agentic systems, several operational challenges remain for production deployment.

A. Key Lifecycle and Revocation

The management of PQC key pairs (ML-DSA) requires a robust Public Key Infrastructure (PKI). Due to the potential for lattice-based cryptanalysis to improve over time, we recommend shorter key rotation cycles (e.g., 6 months) compared to traditional RSA. Furthermore, the orchestrator's private keys should be protected within a Hardware Security Module (HSM) or a Trusted Execution Environment (TEE) like Intel SGX or AWS Nitro Enclaves. While our software-based PQC implementation is portable, hardware-backed isolation is essential to prevent the theft of the PQC identity itself.

B. Standardization and Agility

The IETF is currently standardizing PQC algorithms for JOSE and COSE [12], specifically focusing on URL-safe representations and hybrid signature formats. Our implementation’s shift toward URL-safe Base64 and a four-part token structure anticipates these standards. Developers should maintain *cryptographic agility*, ensuring that the system can transition to newer NIST standards (e.g., Falcon or SPHINCS+) if ML-DSA is found to have unexpected vulnerabilities in the future.

C. Inference vs. Security Overhead

Although our evaluation confirms that the signing latency is minor compared to LLM inference, high-frequency “small” tool calls (e.g., mathematical operations) may experience a higher relative overhead. In such cases, the Context-Adaptive Security Scaling (CASS) is essential to balance security requirements with execution speed.

IX. DISCUSSION: THE PATH TO SOVEREIGN AI AUTONOMY

A. Temporal Integrity and the Historical Root of Trust

The use of ML-DSA ensures that an agent’s identity and its decision-making history are not only sovereign but also future-proof. By binding every action to a PQC-signed identity, we establish a *Historical Root of Trust*. This directly mitigates the “SNDL-driven historical revisionism” threat, where a quantum-capable attacker harvests classically-signed logs today to forge a false reasoning path in the future. By enforcing PQC signatures on every audit entry, we ensure that the “historical truth” of the agent’s actions is preserved against the transition to the quantum era.

B. Quantum Migration Strategy and Hybrid-Post-Quantum Resilience

A direct “switch” to PQC is risky. We advocate for a *Hybrid-Post-Quantum Resilience* (HPQR) approach. By utilizing tokens that wrap both RSA and ML-DSA signatures, the system remains compatible with current Zero Trust infrastructures (e.g., standard API gateways) while achieving the “quantum safety” property required for long-term deployment.

C. Sovereign AI Governance and DAOs

As AI agents move toward decentralized autonomous organizations (DAOs), the immutability of audit logs becomes paramount. Our *PQC-Audit-Chain* ensures that even with the advent of large-scale quantum computers, the history of agent decisions remains cryptographically verifiable. This provides a foundational layer for “AI Sovereignty,” where an agent’s reasoning is independent of any single provider and remains auditable through its verifiable reasoning trace.

X. CONCLUSION

Securing AI agents against quantum threats is a prerequisite for their deployment in critical infrastructure and decentralized governance. Our proposed PQC framework demonstrates that migrating to NIST-standardized quantum-resistant algorithms is practical for distributed agent protocols like MCP. By implementing *Temporal Integrity Proofs* via the *PQC-Audit-Chain*, we provide a mechanism for establishing long-term, immutable reasoning trails that remain verifiable even in the presence of future quantum-capable adversaries. The introduction of *Cryptographic Agility* allows these systems to scale performance and security dynamically, ensuring that the transition to the post-quantum era does not compromise agentic reasoning efficiency. This work establishes a foundational roadmap for building truly sovereign, future-proof autonomous systems that can operate with integrity in a quantum-capable world.

ACKNOWLEDGMENT

This research builds upon the foundational zero-trust principles implemented in the `llm-cli` open-source project.

REFERENCES

- [1] National Institute of Standards and Technology (NIST), “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard,” Aug 2024.
- [2] National Institute of Standards and Technology (NIST), “FIPS 204: Module-Lattice-Based Digital Signature Standard,” Aug 2024.
- [3] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” NIST Special Publication 800-207, 2020.
- [4] Y. Ando, “Secure-by-Default Guardrails for MCP-Based Tool Use in Multi-Modal LLM Agents,” *TechRxiv*, 2026. DOI: 10.36227/techrxiv.177006109.97957916/v1.
- [5] Y. Ando, “Zero-Trust Architecture for MCP-Based AI Agents: A Unified CLI Approach,” *TechRxiv*, 2026. DOI: 10.36227/techrxiv.177155647.75363542/v1.
- [6] Anthropic, “Model Context Protocol (MCP) Specification,” [Online]. Available: <https://modelcontextprotocol.io/> [Accessed: Feb 2026].
- [7] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” arXiv:2210.03629, 2022.
- [8] X. Zhang et al., “Agentic Security: Risks and Protections of Autonomous AI Systems,” *IEEE Security & Privacy*, vol. 23, no. 1, 2025.
- [9] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” *Proceedings 35th Annual Symposium on Foundations of Computer Science*, 1994.
- [10] V. Lyubashevsky et al., “CRYSTALS-Dilithium Algorithm Specifications,” NIST PQC Project, 2023.
- [11] NIST, “Post-Quantum Cryptography Standardization Project,” [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography>.
- [12] P. Prorok et al., “Post-Quantum Signature Algorithms for JOSE and COSE,” IETF draft-ietf-jose-pqc-algs-02, 2024.
- [13] H. Birkholz et al., “Remote Attestation ProcedureS (RATS) Architecture,” RFC 9334, 2023.
- [14] B. Laurie et al., “Certificate Transparency,” RFC 6962, 2013.
- [15] D. Dhinakaran et al., “A post-quantum security architecture for AI-driven surveillance systems: ensuring data protection in the quantum computing era,” *The Journal of Supercomputing*, 2026. DOI: 10.1007/s11227-026-08315-w.